Aula 28: Estilização em Componentes React

Felipe Augusto Rocha da Silva

Desenvolvimento Front End em React

Programa Capacita

INSTITUTO FEDERAL
Minas Gerais

Índice

- INTRODUÇÃO
- INLINE STYLING
- CSS EXTERNO
- PROBLEMAS DO CSS GLOBAL
- CSS MODULES
- STYLED COMPONENTS
- CSS-IN-JS
- TAILWIND CSS
- COMPARATIVO FINAL
- EXERCÍCIOS

Introdução

- CSS pode ser adicionado na aplicação
- Existem muitas maneiras de estilizar o React com CSS
- As três mais comuns são:
  - Inline styling
  - CSS stylesheets
  - CSS Modules

Inline Styling

- Para estilizar um elemento com Inline Styling, o valor deve ser um objeto JavaScript

```
import { createRoot } from 'react-dom/client';

const Header = () => {
  return (
    <>
      <h1 style={{color: "red"}}>Hello
Style!</h1>
      <p>Add a little style!</p>
    </>
  );
}

createRoot(document.getElementById('root')).rende
r(
  <Header />
);
```

Exemplo prático

ARQUIVO HTML index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <script type="text/babel" src="main.jsx"></script>
</head>
<body>
    <div id="root"></div>
</body>
</html>
```

Exemplo prático

ARQUIVO JSX

main.jsx

```
const Header = () => {
  return (
    <>
      <h1 style={{color: "red"}}>Hello Style!</h1>
      <p>Add a little style!</p>
    </>
  );
}

ReactDOM.render(<Header />,
document.getElementById('root'));
```

Objeto JavaScript

- É possível criar um objeto JavaScript com informação de estilo
- Esse objeto deve ser referenciado dentro do atributo style
- Como o Inline CSS é escrito em objeto JavaScript, propriedades com separação em hífen (background-color por exemplo) devem ser escritos em camel case

background-color => backgroundColor

Exemplo prático

ARQUIVO HTML index2.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <script type="text/babel" src="main2.jsx"></script>
</head>
<body>
    <div id="root"></div>
</body>
</html>
```

Exemplo prático

ARQUIVO JSX

main2.jsx

```
const Header = () => {
  const myStyle = {
    color: "white",
    backgroundColor: "DodgerBlue",
    padding: "10px",
    fontFamily: "Sans-Serif"
  };
  return (
    <>
      <h1 style={myStyle}>Hello Style!</h1>
      <p>Add a little style!</p>
    </>
  );
}

ReactDOM.render(<Header />, document.getElementById('root'));
```

CSS externo

- O estilo CSS pode ser escrito em um arquivo separado
- Basta salvar o arquivo com a extensão .css e importá-lo na aplicação
- Para importar, deve-se utilizar o comando import

```
import './MyStylesheet.css';

const Header = () => {
  return (
    <>
      <h1>Hello Style!</h1>
      <p>Add a little style!.</p>
```

Exemplo prático

ARQUIVO HTML index3.html

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <script type="text/babel" src="main3.jsx"></script>
    <link rel="stylesheet" href="MyStylesheet.css">
</head>
<body>
    <div id="root"></div>
</body>
</html>
```

Exemplo prático

ARQUIVO CSS

MyStylesheet.css

```
body {
  background-color: DodgerBlue;
  color: white;
  padding: 40px;
  font-family: Sans-Serif;
  text-align: center;
}
```

Exemplo prático

ARQUIVO JSX

main3.jsx

```
//import 'MyStylesheet.css';

const Header = () => {
  return (
    <>
      <h1>Hello Style!</h1>
      <p>Add a little style!.</p>
    </>
  );
}

ReactDOM.render(<Header />, document.getElementById('root'));
```

Problemas do CSS Global

- A evolução da estilização em React resolve problemas de escopo e manutenção
- A estilização tradicional em CSS enfrenta desafios de especificidade e conflitos globais em aplicações de larga escala
- O ecossistema React evoluiu para abordagens que priorizam a modularidade
- O CSS Global permite vazamento de estilos, dificuldade em remover código morto e conflitos de nomes de classes em projetos grandes

Problemas do CSS Global

ABORDAGENS MODERNAS

- Escopo local (CSS Modules), CSS-in-JS (Styled Components) e Utility-First (Tailwind CSS)

IMPACTO NO PROJETO

- A escolha da técnica impacta diretamente a escalabilidade, a performance e a experiência do desenvolvedor (DX)

CSS Modules

- Garante escopo local sem abandonar a sintaxe tradicional do CSS
- **Escopo Local por Padrão**: Transforma nomes de classes em identificadores únicos durante o build, evitando vazamento de estilos entre componentes
- **Poder total do CSS**: Permite o uso de todo o poder do CSS puro (media queries, pseudo-classes) com a segurança do escopo local
- **Integração Nativa**: Nativo em ferramentas como Vite e Create React App. Ideal para equipes que preferem separar estrutura (JSX) de estilo (CSS)

CSS Modules

IMPLEMENTAÇÃO PRÁTICA

1. **Convenção de nomes**: Arquivos de estilo devem usar a extensão .module.css para serem processados corretamente

2. **Importação Segura**: O estilo é importado como um objeto JavaScript, onde cada chave mapeia para uma classe única

3. **Aplicação no JSX**: Utilize a propriedade **className** acessando a chave correspondente do objeto importado

CSS Modules

IMPLEMENTAÇÃO PRÁTICA

```
// Button.module.css
.button {
  background-color: #0055FF;
  color: white;
  padding: 10px 20px;
  border: none;
}
```

```
// Button.jsx
import React from 'react';
import styles from './Button.module.css';

function Button() {
  return (
    <button className={styles.button}>
      Clique Aqui
    </button>
  );
}
```

Exemplo prático

ARQUIVO HTML

index4.html

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <script type="text/babel" src="main4.jsx"></script>
    <link rel="stylesheet" href="my-style.module.css">
</head>
```

Exemplo prático

ARQUIVO CSS

my-style.module.css

```
.bigred {
  color: Tomato;
  padding: 40px;
  font-family: Sans-Serif;
  text-align: center;
}
```

Exemplo prático

ARQUIVO JSX

main4.jsx

```
const Car = () => {
  return <h1 className="bigred">Hello Car!</h1>;
}

ReactDOM.render(<Car />, document.getElementById('root'));
```

Exemplo prático 2 - Múltiplas Classes

ARQUIVO HTML

index5.html

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <script type="text/babel" src="main5.jsx"></script>
    <link rel="stylesheet" href="my-style.module.css">
</head>
```

Exemplo prático 2 - Múltiplas Classes

ARQUIVO CSS

Button.module.css

```
.mybutton {
  padding: 10px 20px;
  border: none;
  border-radius: 4px;
  cursor: pointer;
}

.primary {
  background-color: #007bff;
  color: white;
}

.secondary {
  background-color: #6c757d;
  color: white;
}
```

Exemplo prático 2 - Múltiplas Classes

ARQUIVO JSX

main5.jsx

```
function App() {
  return (
    <div>
      <button className="mybutton primary">
        My Primary Button
      </button>
      <br/>
      <br/>
      <button className="mybutton secondary">
        My Secondary Button
      </button>
    </div>
  );
}

ReactDOM.render(<App />, document.getElementById('root'));
```

Styled Components

PARADIGMA CSS-IN-JS

- Utiliza tagged template literals para definir estilos diretamente no JavaScript, unindo lógica e visual em um único local
- Cria componentes que já nascem com estilos atrelados, eliminando a necessidade de gerenciar nomes de classes manualmente
- Facilita a manutenção ao manter o estilo próximo à lógica e oferece suporte nativo a temas via React Context

CSS-in-JS

- CSS-in-JS é uma técnica de estilização que permite escrever CSS diretamente no código JavaScript.
- Essa abordagem permite:
  - Escrever CSS usando JavaScript
  - Criar estilos com escopo de componente
  - Usar estilos dinâmicos baseados em props
  - Evitar conflitos de nomes de classes CSS

CSS-in-JS

- CSS-in-JS não faz parte da biblioteca principal do React
- Mas pode ser instalado usando diversas ferramentas de build do React, como Vite, Webpack ou Create React App
- Para instalar styled-components, execute o seguinte comando:

```
npm install styled-components
```

- No caso da nossa aula, vamos utilizar uma script no arquivo html:

```
<script src="https://unpkg.com/styled-components/dist/styled-components.min.js"></script>
```

Exemplo prático

ARQUIVO HTML

index6.html

```
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document</title>
    <script src="https://unpkg.com/react@18/umd/react.development.js"></script>
    <script src="https://unpkg.com/react-dom@18/umd/react-dom.development.js"></script>
    <script src="https://unpkg.com/@babel/standalone/babel.min.js"></script>
    <script type="text/babel" src="main6.jsx"></script>
   <script src="https://unpkg.com/styled-components/dist/styled-components.min.js"></script>
</head>
```

Exemplo prático

ARQUIVO JSX

main5.jsx

```
const MyHeader = styled.h1`
  padding: 10px 20px;
  background-color: #007bff;
  color: white;
`;

function App() {
  return (
    <>
      <MyHeader>Welcome!</MyHeader>
    </>
  );
}

ReactDOM.render(<App />, document.getElementById('root'));
```

Tailwind CSS

- Acelera o desenvolvimento através de uma abordagem utility-first
- **Conceito Utility-First:** Fornece classes utilitárias de baixo nível que permitem construir designs complexos diretamente no JSX, sem sair do arquivo
- **Produtividade:** Elimina a necessidade de criar nomes de classes e alternar constantemente entre arquivos CSS e JS durante o desenvolvimento
- **Sistema de Design consistente:** Altamente configurável via tailwind.config.js, garantindo que toda a equipe utilize a mesma paleta de cores e espaçamentos

Tailwind CSS

- Para ser utilizado na nossa aula, precisaremos inserir uma script no HTML

```
<script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
```

- Essa forma de utilização é apenas didática
- Não deve ser utilizada para produção

Tailwind CSS

- Exemplo de uso - balão de notificação

exempoTailwind.html

```
<!-- container da página -->
<div class="w-screen h-screen bg-slate-100">
 <!-- balão de notificação -->
 <div class="p-4 max-w-sm mx-auto bg-white rounded-xl shadow-lg">
   <div>
     <h3 class="text-xl font-medium text-black">Notification</h3>
     <p class="text-slate-500">You have a new message!</p>
   </div>
 </div>
</div>
```

Tailwind CSS

```
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <script src="https://cdn.jsdelivr.net/npm/@tailwindcss/browser@4"></script>
</head>
<body>
  <!-- container da página -->
  <div class="w-screen h-screen bg-slate-100">
  <!-- balão de notificação -->
      <div class="p-4 max-w-sm mx-auto bg-white rounded-xl shadow-lg">
          <div>
          <h3 class="text-xl font-medium text-black">Notification</h3>
          <p class="text-slate-500">You have a new message!</p>
          </div>
      </div>
  </div>
</body>
```

Comparativo final

CRITÉRIO	CSS MODULES	STYLED COMPONENTS	TAILWIND CSS
Sintaxe	CSS Padrão (.css)	CSS-in-JS (Template Literals)	Classes Utilitárias (HTML/JSX)
Escopo	Local (Hashed Classes)	Local (Dynamic Classes)	Global (Utility Classes)
Dinamicidade	Limitada (Static)	Alta (Props/JS)	Média (Conditional Classes)
Melhor Uso	Projetos grandes com CSS tradicional.	Sistemas de design com temas complexos.	Prototipagem rápida e produtividade.

Exercício 1

- Desenvolver um componente de "Card" reutilizável contendo uma imagem, um título e uma descrição curta
- Criar o arquivo de estilos Card.module.css
- Importar os estilos como um objeto no componente React
- Aplicar classes para o container, imagem e tipografia
- Garantir que não haja vazamento de estilos para outros elementos
- **Desafio Extra**: Implementar um estado de hover no arquivo CSS que altere a cor de fundo do card de forma suave, utilizando variáveis CSS para as cores.

Exercício 2

- Converter o componente "Card" desenvolvido anteriormente para utilizar a biblioteca Styled Components
- Criar componentes estilizados individuais para Container, Title e Description
- Remover todas as referências a arquivos CSS externos e classes manuais
- **Desafio Extra**: Implementar uma prop variant que altere dinamicamente o esquema de cores do card entre os modos 'light' e 'dark'

Exercício 3 - Interface Responsiva

- Criar uma seção de "Grid de Cards" que se adapte perfeitamente a diferentes tamanhos de tela utilizando apenas classes utilitárias
- Utilizar classes de Grid ou Flexbox do Tailwind para o layout
- Aplicar classes de espaçamento (gap, padding, margin) consistentes
- Configurar a tipografia e cores usando o sistema de design do Tailwind
- Garantir que o layout seja mobile-first
- **Desafio Extra**: Fazer com que o grid exiba 1 coluna no mobile, 2 colunas no tablet (md) e 3 colunas no desktop (lg)